Concurso 108 – nivel 3 Por SyXe'05

CracksLatinoS! 2006

-* UnPackMe_VBOWatch Protector 2.0 *-

Programa	**UnPackMe_VBOWatch Protector 2.0**	
Download	FTP de Ricardo	
Descripción	UnPackMe del concurso 108 nivel3	
Herramientas	OllyDbg 1.10 + ImpREC 1.6f + PEiD 0.94 + RDG Packer Detector 0.64 BETA	
Dificultad	Muy fácil	
Compresor/Compilador		
Protección	Archivo empaketado	
Objetivos	Desempacar a mano	
Cracker	**SyXe'05**	**Fecha:** 03/11/06
Tutorial n°	3ª solución de un concurso	

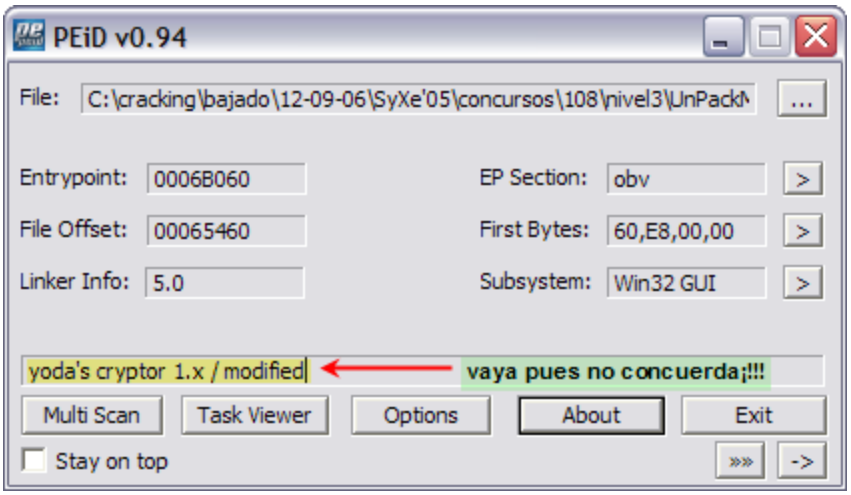
-** Introduccion **-

Hola a todos los CracksLatinos!

Un saludo a todos ellos, en esta ocasión nos volvemos a ver las caras para tratar de solucionar un concurso, se trata del con curso108 N3, y es un UnPackMe empaquetado con VBO Watch Protection v2.0, un packer que jamás antes vi, por lo que no tengo mucho que decir acerca de él. Todo ello lo iremos viendo sobre la marcha a lo largo del tutorial :D

-** Explorando el objetivo **-

Lo primero que debemos hacer como todo buen cracker ordenado es abrir el ejecutable protegido "UnPackMe_VBOWatch Protector 2.0.exe" en un analizador de archivos tipo PeiD o RDG Packer Detector. En este caso probamos con PEiD 0.94 y nos dice esto:



Miramos con RDG Packer Detector v0.6.4 BETA y nos dice:

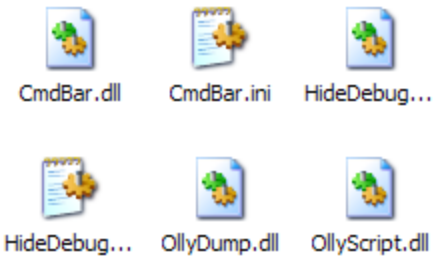
Ambos analizadores sacan lo mismo, si lo paso al modo M-B me dice "NADA", pero si miramos los objetivos de este nivel 108 veríamos esto:

3) Desempakar VBO watch

Bueno ante las dudas nos fiaremos de Ricardo que fue quien envió el concurso y allí dice que es "VBO watch" en cualquier caso pasamos al atake YA, jeje..

-** Desempacando **-

Cerramos ambos analizadores y nos abrimos un Ollydbg con los siguientes plugins:



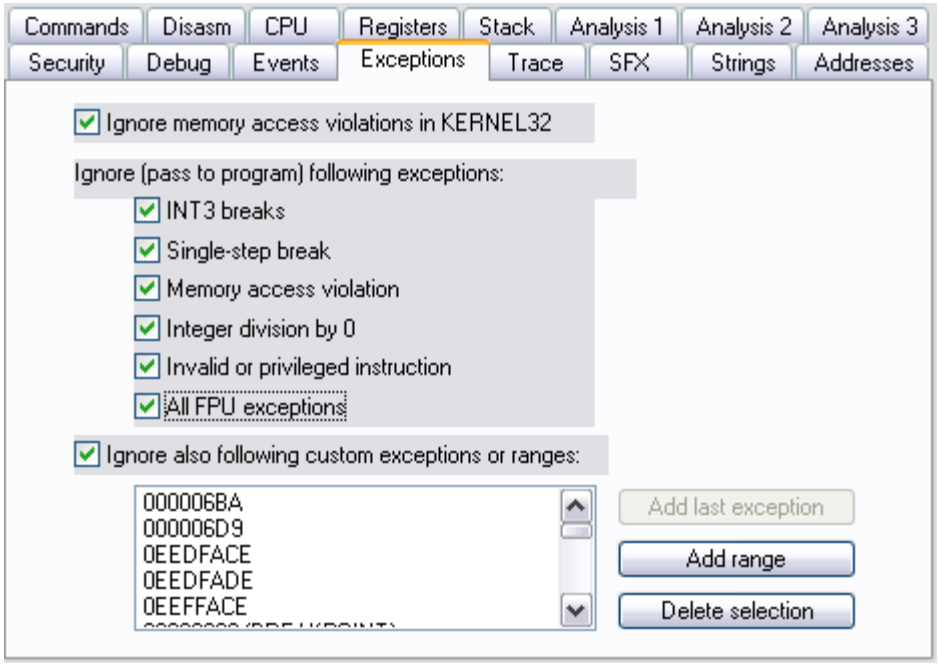
Esperemos que sean suficientes, si hiciera falta alguno más pues lo colocaremos sobre la marcha.

Abrimos un Olly y queda el packer detenido en el EP:

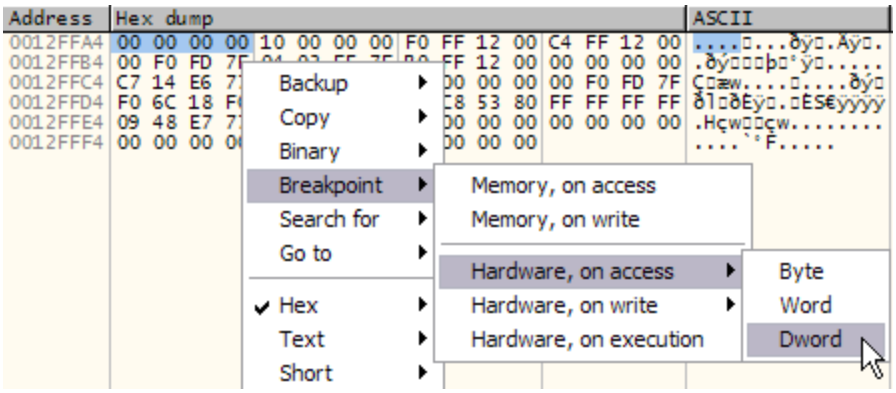
```
Address   Hex dump        Disassembly                Comment
0046B060  60             pushad
0046B061  E8 00000000    call    0046B066
0046B066  5D             pop     ebp
0046B067  81ED 4F204000  sub     ebp,0040204F
0046B06D  B9 30090000    mov     ecx,930
0046B072  8DBD 97204000  lea     edi,[ebp+402097]
0046B078  8BF7           mov     esi,edi
0046B07A  AC             lodsb
0046B07B  C0C0 83        rol     al,83                Shift constant out of ran
0046B07E  2AC1           sub     al,cl
0046B080  2C B8          sub     al,0B8
0046B082  EB 01          jmp     short 0046B085
0046B084  E8 048AC0C8    call    C9073A8D
0046B089  A3 C0C8E7EB    mov     [EBE7C8C0],eax
```

Traceamos una sola línea con F7/F8 para que se ejecute el PUSHAD y miramos en la ventanita de registros, se puede ver que ESP está de color ROJO, esto es porque tras el PUSHAD se modificó, entonces lo seleccionamos y picamos con el botón derechosobre él y en el menú emergente le damos a "Follow in Dump"

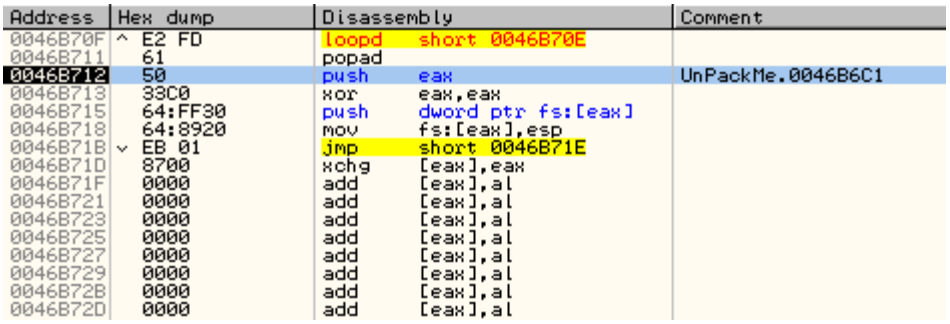
Hecho esto vamos a la ventanita dump y seleccionamos los 4 bytes que quedaron a la vista (los de ESP) y le ponemos un HBP on Memory Access, primero configuramos Olly para que no se detenga en ninguna excepción:



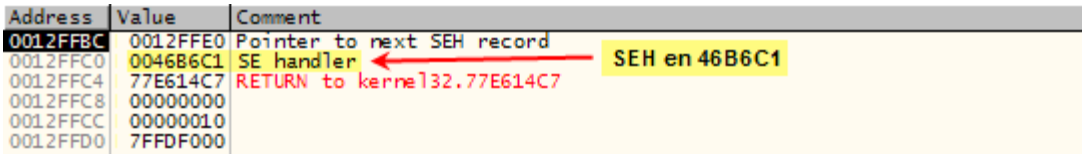
Así si ocurre alguna excepción Olly mismo la pasará sin darnos el control a nosotros, y no olvidar poner el HBP on Access en ESP:



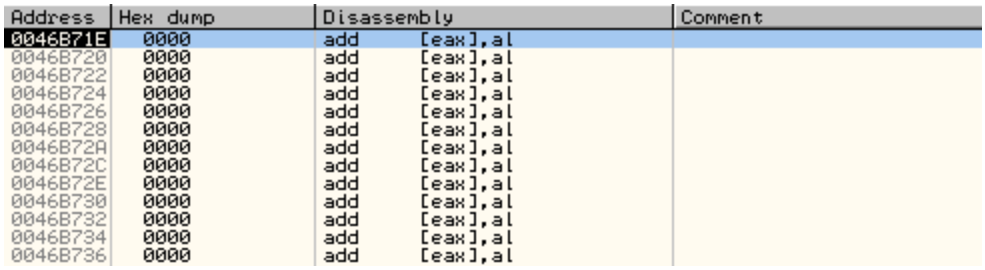
Ahora doy RUN y Olly se detiene aquí:



Se ve el típico POPAD y luego unas líneas, las vamos traceando con F8 mismo ya que no influye hacerlo con F7 u F8 puesto que no hay ningún CALL entre ellas, lo que hace es instalar un manejador de excepciones (SEH) que podemos ver en la zona de pila (stack):



Luego el siguiente JMP nos pone en una zona llena de 00s que por su puesto es zona inválida y que no podrá ejecutar, para qué hace esto? Bien sencillo, para generar una excepción ACCESS_VIOLATION y el SEH tome el control, puesto que nos dejó aquí:



así que ponemos un BP al inicio del mismo:

```
Address   Hex dump        Disassembly                  Comment
0046B6C1  55              push    ebp                  inicio SEH
0046B6C2  8BEC            mov     ebp,esp
0046B6C4  57              push    edi
0046B6C5  8B45 10         mov     eax,[ebp+10]
0046B6C8  8BB8 C4000000   mov     edi,[eax+C4]
0046B6CE  FF37            push    dword ptr [edi]
0046B6D0  33FF            xor     edi,edi
0046B6D2  64:8F07         pop     dword ptr fs:[edi]
0046B6D5  8380 C4000000   add     dword ptr [eax+C4],8
0046B6DC  8BB8 A4000000   mov     edi,[eax+A4]
0046B6E2  C1C7 07         rol     edi,7
0046B6E5  89B8 B8000000   mov     [eax+B8],edi
0046B6EB  B8 00000000     mov     eax,0
0046B6F0  5F              pop     edi
0046B6F1  C9              leave
0046B6F2  C3              ret
```

Apretamos F9 y estamos ahí:

```
Address   Hex dump        Disassembly                  Comment
0046B6C1  55              push    ebp                  inicio SEH
0046B6C2  8BEC            mov     ebp,esp
0046B6C4  57              push    edi
0046B6C5  8B45 10         mov     eax,[ebp+10]
0046B6C8  8BB8 C4000000   mov     edi,[eax+C4]
0046B6CE  FF37            push    dword ptr [edi]
```

Si traceamos desde aquí veremos cómo saltará a la librería ntdll.dll de sistema, y si continuamos haviéndolo llegará un momento en que llegamos a la llamada a la ZwContinue:

```
Address   Hex dump        Disassembly                  Comment
77FA4DBD  0AC0            or      al,al
77FA4DBF  74 0C           je      short 77FA4DCD
77FA4DC1  5B              pop     ebx
77FA4DC2  59              pop     ecx
77FA4DC3  6A 00           push    0
77FA4DC5  51              push    ecx
77FA4DC6  E8 480BFCFF     call    ZwContinue
77FA4DCB  EB 0B           jmp     short 77FA4DD8
```

A partir de aquí tenemos como mínimo dos opciones, una sería seguir la excepción traceando, esto tiene el inconveniente de que llegará un momento en que no la podremos seguir desde Olly pues salta a ring0, primero saltará a:

```
77F65913 >  B8 20000000     mov     eax,20
77F65918    BA 0003FE7F     mov     edx,7FFE0300
77F6591D    FFD2            call    edx
77F6591F    C2 0800         ret     8
```

Y ese CALL EDX nos deja aquí:

```
Address   Hex dump        Disassembly                  Comment
7FFE0300  8BD4            mov     edx,esp
7FFE0302  0F34            sysenter
7FFE0304  C3              ret
7FFE0305  8BD4            mov     edx,esp
7FFE0307  0F05            syscall
7FFE0309  C3              ret
```

Ese SYSENTER no se puede seguir con Olly puesto que salta a ring0 donde Olly no tiene privilegios, entonces podríamos poner UN BPM en la sección de código del programa para retornar allí una vez deje de ejecutar código en ring0, pero hay algo más interesante y es que estando parados aqui:

```
Address   Hex dump        Disassembly                  Comment
77FA4DBD  0AC0            or      al,al
77FA4DBF  74 0C           je      short 77FA4DCD
77FA4DC1  5B              pop     ebx
77FA4DC2  59              pop     ecx
77FA4DC3  6A 00           push    0
77FA4DC5  51              push    ecx
77FA4DC6  E8 480BFCFF     call    ZwContinue
77FA4DCB  EB 0B           jmp     short 77FA4DD8
```

Si mirábamos al Dumo veíamos estos valores:

Concurso 108 - nivel 3 Por SyXe'05

Address	Value	Comment
0012FCCC	0012FCF0	Arg1 = 0012FCF0 ← vamos a esta dirección!
0012FCD0	00000000	Arg2 = 00000000
0012FCD4	C0000005	← código de excepción
0012FCD8	00000000	
0012FCDC	00000000	ACCESS_VIOLATION (0x0C000005h)
0012FCE0	0046B71E	UnPackMe.0046B71E

Hacemos un "Follow in dump" a la dirección de Arg1 en este caso 0x12FCF0 y allí hacemos un pequeño scroll hacia abajo para ver esto:

Address	Hex dump	ASCII
0012FCF0	4F 00 01 00 A4 FF 12 00 00 00 00 00 00 00 00 00	?.□.¤ÿ□.....
0012FD00	00 00 00 00 00 00 00 00 00 00 00 00 7F 02 FF FF	.....□□ÿÿ
0012FD10	20 40 FF FF FF FF FF FF 76 18 0A 78 1B 00 56 00	@ÿÿÿÿÿÿv□.x□.V.
0012FD20	B0 21 09 78 23 00 FF FF E6 17 F4 77 78 17 F4 77	°!.x#.ÿÿæ□ôwx□ôw
0012FD30	B2 17 78 92 56 00 00 00 00 [illegible] 00 00 00	²□x'V.....l...
0012FD40	6C 00 6C 00 00 00 C4 CD 12 [illegible] 77 9A 0A	l.l...ÄÍ□.h:Ñwš.
0012FD50	00 00 00 00 F4 CE 12 00 32 [illegible] 00 0C CE	....ôÎ□.2á.....Î
0012FD60	12 00 32 E1 00 00 00 00 00 [illegible] 3F 00 00	□.2á.....€ÿ?..
0012FD70	00 00 00 00 00 80 FF 3F 00 [illegible] 00 00 00	.....€ÿ?.....
0012FD80	38 00 00 00 23 00 00 00 23 00 00 00 00 00 00 00	8...#...#.....
0012FD90	10 00 00 00 E3 84 00 60 04 03 FE 7F B0 FF 12 00	□...ã„.`□□þ□°ÿ□.
0012FDA0	00 00 00 00 F0 FF 12 00 B0 71 42 00 1B 00 00 00	....ðÿ□.°qB.□...
0012FDB0	46 03 01 00 C4 FF 12 00 23 00 00 00 7F 02 20 40	F□□.Äÿ□.#...□□ @
0012FDC0	00 00 56 00 76 18 0A 78 1B 00 00 00 B0 21 09 78	..V.v□.x□...°!.x
0012FDD0	23 00 FF FF 80 1F 00 00 FF FF 00 00 E6 17 F4 77	#.ÿÿ€□..ÿÿ..æ□ôw

El OEP¡!!

Ese 4271B0 no es ni más ni menos que nuestro OEP y a la direción que retornará tras pasar por la llamada a ZwContinue, tenemos dos alternativas, bien poner un BP con F2 allí o bien poner un BPM en toda la sección de código, con cualquiera de estos métodos nos plantamos en todo el OEP X)

Address	Hex dump	Disassembly		Comment
004271B0	55	push	ebp	
004271B1	8BEC	mov	ebp,esp	
004271B3	6A FF	push	-1	
004271B5	68 600E4500	push	00450E60	
004271BA	68 C8924200	push	004292C8	
004271BF	64:A1 00000000	mov	eax,fs:[0]	
004271C5	50	push	eax	
004271C6	64:8925 000000	mov	fs:[0],esp	
004271CD	83C4 A8	add	esp,-58	
004271D0	53	push	ebx	
004271D1	56	push	esi	
004271D2	57	push	edi	
004271D3	8965 E8	mov	[ebp-18],esp	
004271D6	FF15 DC0A4600	call	[460ADC]	kernel32.GetVersion
004271DC	33D2	xor	edx,edx	

Ya lo tenemos:

OEP==4271B0

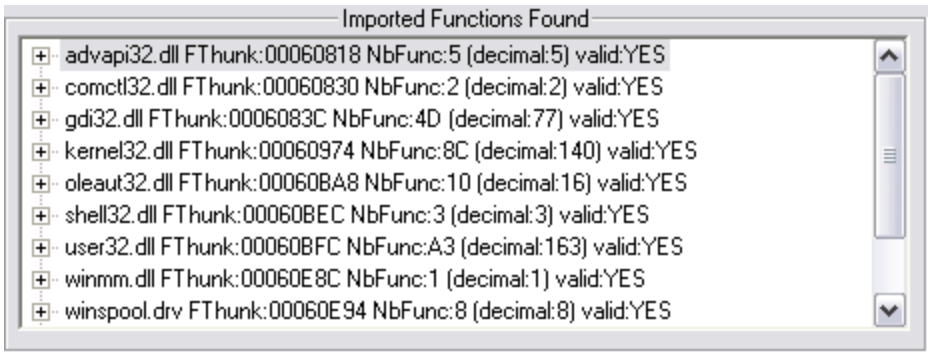
De todos modos el UnPackMe de Teddy Roggers siempre tiene el OEP aquí (lógico) pero igual aprendimos a movernos por nosotros mismos por la ZwContinue queno va nada mal para otras veces en que no está tan claro al módulo y/o sección a la que saltará.

Una vez descubierto este valor el resto es coser y cantar, jeje.. dumpeamos co el plugin OllyDump que rula perfecto en este caso, sólo hay que destildar la opción "Rebuild Imports" para después hacerlo nosotros con el ImpREC, picamos en "Get EIP as OEP" una vez y luego en "Dump" a este archivo lo llamaremos por ejemplo dumped.exe.

Ahora sí abrimos el ImpREC y seleccionamos el programa de la lista donde dice "Attach to an Activeprocess" en el campo OEP ponemos:

4271b0 - 400000 = 271B0 /* su offset*/

Y picamos en IAT Autosearch, nos saca la IAT correctamente, damos a "Get Imports" y nos saca todas las importaciones correctamente, sólo queda picar en "Fix Dump" para reparar dumped.exe:

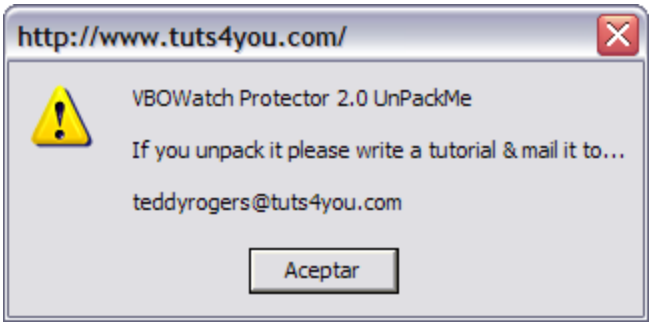


Todas las importaciones correctas, a ver que dice tras el "Fix Dump":

```
Fixing a dumped file...
C (decimal:12) module(s)
1B9 (decimal:441) imported function(s).
*** New section added successfully. RVA:0006D000 SIZE:00002000
Image Import Descriptor size: F0; Total length: 1F1E
C:\cracking\bajado\12-09-06\SyXe'05\concursos\108\nivel3\dumped_.exe saved succ
```

Vale dejo un archivo llamado "dumped_.exe" en la misma carpeta donde está el unPackMe, en un principio no nos debería dar problemas de ejecución.

Sólo queda probarlo, le hacemos doble clic (a dumped_.exe) y nos llevamos una más que grata sorpresa XDDDDD



El UnPackMe echa a correr porque lo tenemos ya desempakadito y bien limpito de packer que era justo lo que se pedía en este nivel, así pues "prueba superada"

-** Saludos y/o Agradecimientos **-

Y ahra que terminamos el tutecito aprovecharé para dar un saludete a mis grandes amigos de la lista CracksLatinoS, en especial a:

Absolom, solid, Ricardo, Red@Hawk, Morales, Juan Jose, txino_crk, emadicius, +NCR, Agujeta, marciano, MairJr, stzwei (suei), DeAtH, dapaf, GUAN, c40s, k0rt (desaparecido ¿?), Alfonso_iat, Ulaterck, Yllera, Arapumk, apuromafo, Mitaka, Carlos y todo el grupo CracksLatinoS!

No me tiren piedras aquellos de los que me olvidé temporalmente, voy con las prisas y es normal que ocurra pero por si acaso quedan TODOS saludados :D

Un abrazo y espero seguir escribiendo tutes como venía haciendo hasta ahora..

-** Contacto **-

Concurso 108 - nivel 3 Por SyXe'05

El autor puede ser contactado via e-mail en las siguientes direcciones:

syxe00@yahoo.es
syxe05@gmail.com // mucho mejor escribirme a esta

Todos los e-mails irán siendo contestados. Un abrazo a todos los crackers que dedican su tiempo a escribir tutoriales para que el resto aprendamos :D

hasta la próxima:

SyXe'05
CracksLatinoS